

Code Review Communication Guidelines (The 7 Cs)

Effective code reviews are essential for producing high-quality, maintainable software. Just as importantly, they rely on **clear and respectful communication**. Focused feedback helps reviewers and authors align quickly, reduces unnecessary back-and-forth, and avoids confusion or frustration.

When participating in code reviews (PRs, comments, or discussions), we prioritize **short, direct, and actionable feedback** that captures all essential information. This is generally more effective than long, elaborate, or repetitive explanations, which can obscure the main issue and slow down the review process.

General Principles

- Prefer **clear, concise comments** over long or repetitive explanations.
 - Focus on **improving the code**, not evaluating the person.
 - **Assume positive intent** from both authors and reviewers.
 - Keep discussions **actionable and focused** to minimize back-and-forth.
-

The 7 Cs of Communication (Applied to Code Review)

1. Clear

State the intent of your comment explicitly. Make it obvious whether you are flagging a bug, making a suggestion, asking a question, or noting a style preference.

- Explicitly state the purpose of each comment.
- Avoid ambiguity—do not assume the author will infer intent.
- If something blocks approval, say so clearly.

Good:

> “This will break when X is NULL; we should guard against that.”

Avoid:

> “This feels risky.”

2. Concise

Keep comments focused on the specific change under review.

- Address **one issue per comment**.
- Avoid repeating the same point in different ways.
- If a comment becomes long, summarize or move the discussion elsewhere.

Aim for **signal over verbosity**.

3. Concrete

Be specific and precise so the author can act quickly.

- Reference exact lines, functions, files, or behaviors.
- Provide examples or expected outcomes when suggesting changes.

Example:

> “Line 42 assumes sorted input—should we enforce this or document it?”

4. Correct

Ensure technical accuracy and appropriate terminology.

- Verify assumptions before commenting.
- Use terminology consistent with the codebase and audience.
- Ask questions instead of speculating when unsure.

Correctness builds trust in the review process.

5. Coherent

Structure feedback logically and consistently.

- Separate unrelated issues into distinct comments.
- Keep feedback organized and easy to follow.
- Maintain a consistent, professional tone throughout.

One comment = one concern.

6. Complete

Provide enough context for the author to take action.

- Explain *why* a change matters when suggesting it.
- Clearly indicate required vs. optional changes.
- Make next steps explicit.

Example:

> “Required for merge” vs. “Optional improvement”

7. Courteous

Code reviews are about improving code—not judging people.

- Be respectful, professional, and empathetic.
- Avoid sarcastic, dismissive, or passive-aggressive language.
- Acknowledge good work and improvements when appropriate.

Example:

> “Nice refactor—this makes the logic much clearer.”

Final Notes

- Code reviews are **collaborative by design**.
 - The goal is **shared understanding, better code, and continuous learning**.
 - Clear communication makes reviews **faster, less frustrating, and more effective** for everyone.
-

Authors Antonia Chroni, PhD (@AntoniaChroni)

These tools and pipelines have been developed by the Bioinformatic core team at the St. Jude Children’s Research Hospital. These are open access materials distributed under the terms of the BSD 2-Clause License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original author and source are credited.